



การศึกษาและออกแบบ Algorithm สำหรับการแก้ไขปัญหา
Rastrigin Function ในการหา Global Minimum

ผู้จัดทำ

นางสาวดิษย์ธร สุทธาเวช รหัสนักศึกษา 66340500019

นางสาวบุญยาพร ปรีชาศุทธิ์ รหัสนักศึกษา 66340500031

รายวิชา FRA502 Advanced Robotic Control and Simulation with ROS2 (ARCS2)

สถาบันวิทยาการหุ่นยนต์ภาคสนาม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ประจำปีการศึกษา 1/2568

บทนำ

Global Optimization เป็นปัญหาพื้นฐานที่มีความสำคัญในหลากหลายสาขาวิชา โดยเฉพาะในกรณีที่ฟังก์ชันมีลักษณะซับซ้อนและเป็น Non-Convex ซึ่งมีค่าต่ำสุดเฉพาะที่ (Local Minima) กระจายอยู่หลายจุด การค้นหาค่าต่ำสุดสัมบูรณ์ (Global Minimum) จึงเป็นกระบวนการที่ซับซ้อนและต้องการวิธีการเฉพาะทาง

Rastrigin Function เป็นฟังก์ชันมาตรฐานที่นิยมใช้ในการทดสอบประสิทธิภาพของอัลกอริทึมในการหาค่าที่เหมาะสมที่สุด เนื่องจากมีโครงสร้างที่ซับซ้อนจากการมี Local Minima จำนวนมาก ทำให้วิธีการแบบดั้งเดิมที่อาศัยการคำนวณ Gradient มักติดอยู่ใน Local Minima และไม่สามารถค้นหา Global Minimum ได้ โดยฟังก์ชันนี้มีรูปแบบทั่วไปดังสมการที่ (1)

$$f(x) = An + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)] \quad (1)$$

เมื่อ $x = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, $A = 10$ (ค่าคงที่) และ $n =$ จำนวนมิติ (Dimensions)

ในกรณี 2 มิติ จะเป็นไปดังสมการที่ (2) และมี Global Minimum อยู่ที่ $x^* = (0, 0, \dots, 0)$ ซึ่ง $f(x^*) = 0$

$$f(x_1, x_2) = 20 + x_1^2 + x_2^2 - 10 \cos(2\pi x_1) - 10 \cos(2\pi x_2) \quad (2)$$

ในขอบเขตของการหาค่าที่เหมาะสมที่สุดแบบดั้งเดิม มีแนวคิดพื้นฐาน 2 ประเภทหลัก ได้แก่ การหาค่าที่เหมาะสมที่สุดแบบไม่มีเงื่อนไขบังคับ (Unconstrained Optimization) เช่น Newton's Method และ Damped Newton's Method ที่ปรับปรุงความเสถียรของการเข้าสู่คำตอบโดยไม่มีเงื่อนไขบังคับ และการหาค่าที่เหมาะสมที่สุดแบบมีเงื่อนไขบังคับ (Constrained Optimization) เช่น Quadratic Programming และ Augmented Lagrangian Method ที่สามารถจัดการกับข้อจำกัดต่าง ๆ ได้ ซึ่งวิธีการเหล่านี้มักมีประสิทธิภาพสูงเมื่อฟังก์ชันเป็น Convex หรือมีความราบรื่นเพียงพอ ทำให้เมื่อนำไปประยุกต์ใช้กับปัญหา Non-Convex เช่น Rastrigin Function มักจะประสบปัญหาการติดอยู่ใน Local Minima และไม่สามารถรับประกันการหลีกเลี่ยง Local Minima ได้อย่างสมบูรณ์

เนื่องจากข้อจำกัดของวิธีการแบบดั้งเดิม จึงมีการพัฒนาวิธีการหาค่า Global Minimum ที่ออกแบบมาโดยเฉพาะสำหรับปัญหา Non-Convex ซึ่งสามารถแบ่งออกเป็น 2 รูปแบบ ได้แก่ วิธีการเชิงกำหนด (Deterministic Methods) และวิธีการแบบสุ่ม (Stochastic Methods) ซึ่ง Deterministic Methods มีความสามารถในการรับประกันการค้นพบ Global Minimum ภายในความคลาดเคลื่อนที่กำหนดไว้ (ϵ) โดยอาศัยการสำรวจพื้นที่คำตอบอย่างเป็นระบบ ในขณะที่ Stochastic Methods ใช้กลไกการสุ่มในการสำรวจพื้นที่คำตอบ ทำให้มีความรวดเร็วและสามารถประยุกต์ใช้กับปัญหาที่มีมิติสูงได้ แต่ไม่สามารถรับประกัน Global Optimality ได้

การศึกษานี้จึงมุ่งเน้นการเปรียบเทียบทั้งสองแนวทาง ได้แก่ อัลกอริทึม Branch and Bound (BnB) ที่เป็น Deterministic Methods และอัลกอริทึม Simulated Annealing (SA) ที่เป็น Stochastic Methods เพื่อวิเคราะห์ Trade-off ระหว่างความถูกต้องของผลลัพธ์กับประสิทธิภาพในการคำนวณ

Branch and Bound (BnB) Algorithm

Branch and Bound (BnB) เป็นวิธี Deterministic ที่ใช้หลักการ Divide and Conquer แบ่งพื้นที่คำตอบออกเป็น Box ย่อย ๆ (B) และคำนวณ Lower Bound (LB) กับ Upper Bound (UB) เพื่อตัด Box ที่ไม่มี Global Minimum ทั้งนี้วิธีนี้รับประกันการพบ Global Minimum ที่อยู่ในขอบเขตที่กำหนดทางคณิตศาสตร์ โดยคำตอบจะอยู่ภายในความคลาดเคลื่อน ϵ ที่กำหนด

สำหรับ Box $B = [a_1, b_1] \times [a_2, b_2]$ สามารถคำนวณหาค่า Lower Bound และ Upper Bound ได้ดังสมการที่ (3) และ (4) รวมทั้งมีเงื่อนไขการตัด Box (Pruning Condition) และเงื่อนไขการหยุดแบ่ง Box (Stopping Criterion) ดังสมการที่ (5) และ (6)

$$LB(B) = A_n + \sum_{i=1}^n (b_i - a_i) \left[\min(a_i^2, b_i^2) - A \right] \quad (3)$$

$$UB(B) = f\left(\frac{a+b}{2}\right) \quad (4)$$

$$LB(B) \geq f_{best} \quad (5)$$

$$\max_{(i=1, \dots, n)} (b_i - a_i) < \epsilon \quad (6)$$

กระบวนการของ BnB Algorithm ประกอบด้วย 3 แนวคิดหลัก ดังนี้

1. Branching: กระบวนการแบ่ง Box ออกเป็น 2 ส่วนย่อยตามมิติที่มีความกว้างมากที่สุด โดยแบ่งที่จุดกึ่งกลาง เพื่อเร่งให้ Box มีขนาดเล็กลงอย่างรวดเร็ว ซึ่งเป็นเงื่อนไขการหยุดแบ่ง Box
2. Bounding: กระบวนการคำนวณขอบเขตล่าง (LB) ที่รับประกันว่า $f(x) \geq LB$ สำหรับทุกจุด x ใน Box และขอบเขตบน (UB) ที่ได้จากการประเมินค่าฟังก์ชันที่จุดกึ่งกลางของ Box ซึ่งการคำนวณ LB ที่แม่นยำเป็นปัจจัยสำคัญต่อประสิทธิภาพของ BnB
3. Pruning: กลไกการตัด Box ที่มี LB มากกว่าหรือเท่ากับค่าที่ดีที่สุดที่พบจนถึงปัจจุบัน (f_{best}) หรือเป็นการตัด Box ที่มั่นใจว่าไม่มี Global Minimum ทำให้ช่วยลดจำนวน Box ที่ต้องสำรวจลงได้

กระบวนการทำงานของ BnB เริ่มต้นด้วยการคำนวณ LB และ UB ของ Domain เริ่มต้น จากนั้นเข้าสู่วงจรหลักที่เริ่มด้วยการเลือก Box ที่มี LB ต่ำที่สุดจาก Priority Queue เพื่อตรวจสอบเงื่อนไขการหยุด หากความกว้างสูงสุดของ Box น้อยกว่าค่าความคลาดเคลื่อน ϵ จะบันทึกคำตอบและหยุดแบ่ง Box นั้น หากไม่เข้าเงื่อนไขจะทำ Branching แบ่ง Box ออกเป็น 2 ส่วนตามมิติที่กว้างที่สุด เพื่อคำนวณ LB และ UB ของแต่ละ Box ย่อย โดย UB จะคำนวณจากการประเมินค่าฟังก์ชันที่จุดกึ่งกลางและทุกมุม แล้วจึงทำ Pruning โดยตัด Box ที่มี $LB \geq f_{best}$ เนื่องจากมั่นใจว่าไม่มี Global Minimum และเพิ่ม Box ที่เหลือเข้า Queue เพื่อทำการสำรวจต่อไป

Simulated Annealing Algorithm

Simulated Annealing (SA) เป็นวิธี Stochastic ที่ได้รับแรงบันดาลใจจากกระบวนการชุบแข็งโลหะ (Annealing) ในทางโลหวิทยา ซึ่งเป็นการให้ความร้อนแก่โลหะแล้วค่อย ๆ ลดอุณหภูมิลงอย่างช้า ๆ เพื่อให้โครงสร้างผลึกเรียงตัวอยู่ในสถานะที่มีพลังงานต่ำที่สุด

อัลกอริทึม SA นำหลักการนี้มาประยุกต์ใช้กับปัญหา Optimization โดยใช้ Temperature Parameter (T) ในการควบคุมความน่าจะเป็นในการยอมรับคำตอบที่แย่ลง เพื่อช่วยให้สามารถหลุดจาก Local Minima ได้ โดยจะเริ่มต้นจากการสุ่มจุดเริ่มต้น (x_0) และทำการสำรวจพื้นที่คำตอบ โดยการสร้าง Neighbor Solution (x') ที่อยู่ใกล้เคียงกับคำตอบปัจจุบัน (x) ซึ่งการยอมรับคำตอบใหม่จะขึ้นอยู่กับ 2 เงื่อนไข ดังนี้

- หาก $f(x') < f(x)$: ยอมรับคำตอบใหม่เสมอ เนื่องจากค่าฟังก์ชันดีขึ้น
- หาก $f(x') \geq f(x)$: ยอมรับด้วยความน่าจะเป็นตาม Metropolis Criterion

โดยความน่าจะเป็นในการยอมรับตาม Metropolis Criterion นิยามได้ดังสมการที่ (7)

$$P(\text{accept}) = \{1 \text{ if } \Delta f < 0, e^{(-\Delta f/T)} \text{ if } \Delta f \geq 0\} \quad (7)$$

โดย $\Delta f = f(x') - f(x)$ คือ ความแตกต่างของค่า Objective Function และ T คือ อุณหภูมิ ณ Iteration ปัจจุบัน

อุณหภูมิจะค่อย ๆ ลดลงตามจำนวน Iteration ตาม Cooling Schedule ซึ่งในที่นี้ใช้ Linear Cooling Schedule ดังสมการที่ (8)

$$T(k) = T_0(1 - k/k_{\max}) \quad (8)$$

และการสร้าง Neighbor Solution ทำโดยการเคลื่อนที่แบบสุ่มรอบ ๆ จุดปัจจุบัน ดังสมการที่ (9) และ (10)

$$x'_i = x_i + \text{Uniform}(-s_i, s_i) \quad (9)$$

$$s_i = 0.5 \times (b_i - a_i) \times T(k)/T_0 \quad (10)$$

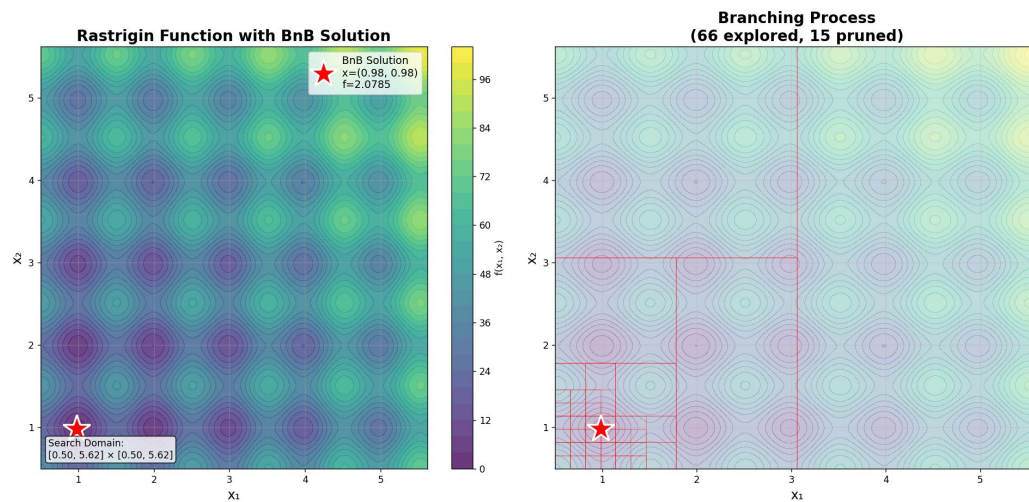
โดยที่ s_i = Step Size สำหรับมิติที่ i และ $[a_i, b_i]$ = ขอบเขตของตัวแปร x_i

สรุปกระบวนการทำงานของ SA เริ่มต้นด้วยการสุ่มจุดเริ่มต้น x_0 ในพื้นที่คำตอบและตั้งค่าอุณหภูมิเริ่มต้น T_0 จากนั้นเข้าสู่วงจรหลักที่ทำซ้ำ $k_{\max}$ ครั้ง เริ่มจากการคำนวณ Temperature ปัจจุบัน $T(k)$ ตาม Linear Cooling Schedule จากนั้นสร้าง Neighbor Solution x' ด้วยการเคลื่อนที่แบบสุ่มรอบ ๆ จุดปัจจุบัน x ด้วย Step Size ที่ลดลงตาม Temperature เพื่อให้การสำรวจมีความหนาแน่นในช่วงแรกและมีความละเอียดในช่วงท้ายที่ใกล้สิ้นสุดการทำงาน แล้วจึงคำนวณความแตกต่าง Δf เพื่อตัดสินใจยอมรับหรือปฏิเสธคำตอบใหม่ตาม Metropolis Criterion ซึ่งทำให้สามารถหลุดพ้นจาก Local minima ได้ในช่วงที่ Temperature สูง สุดท้ายจึงอัปเดต Current Solution ถ้ายอมรับ และอัปเดต Best Solution หากพบคำตอบที่ดีกว่าเดิม

วิธีการทดลอง

ในการทดลองได้เลือกใช้ Offset Domain $[0.5, 5.62]^d$ แทน Standard Domain $[-5.12, 5.12]^d$ โดยรักษารัศมี Domain ไว้เท่าเดิม (5.12 หน่วย) เพื่อให้ Global Minimum ที่แท้จริง $x^* = (0, 0, \dots, 0)$ อยู่นอกขอบเขตแต่อยู่ใกล้ Boundary ของ Domain (ห่าง 0.5 หน่วย) สะท้อนสถานการณ์จริงที่ Optimum อยู่ใกล้ข้อจำกัดทางกายภาพ (Boundary Constraints) รวมทั้งยังทำให้ Best Solution ใน Domain อยู่ที่มุม $x^* \approx (0.5, 0.5, \dots, 0.5)$ ซึ่งบังคับให้ BnB ต้องตรวจสอบทุก Corner (2^d Points) และทดสอบว่า SA สามารถสำรวจ Boundary ได้มีประสิทธิภาพเพียงใด โดยระยะห่าง 0.5 หน่วยถือว่าเป็นระยะที่ไม่มากเกินไปจนเกินไป และไม่ใกล้จนกลายเป็นปัญหาง่าย Domain Design ทำให้สามารถแยกแยะความสามารถของแต่ละอัลกอริทึมได้ชัดเจนทั้งในด้านความถูกต้องและประสิทธิภาพในการขยายสเกล (Scalability) ในปัญหาที่มีข้อจำกัดเชิง Boundary

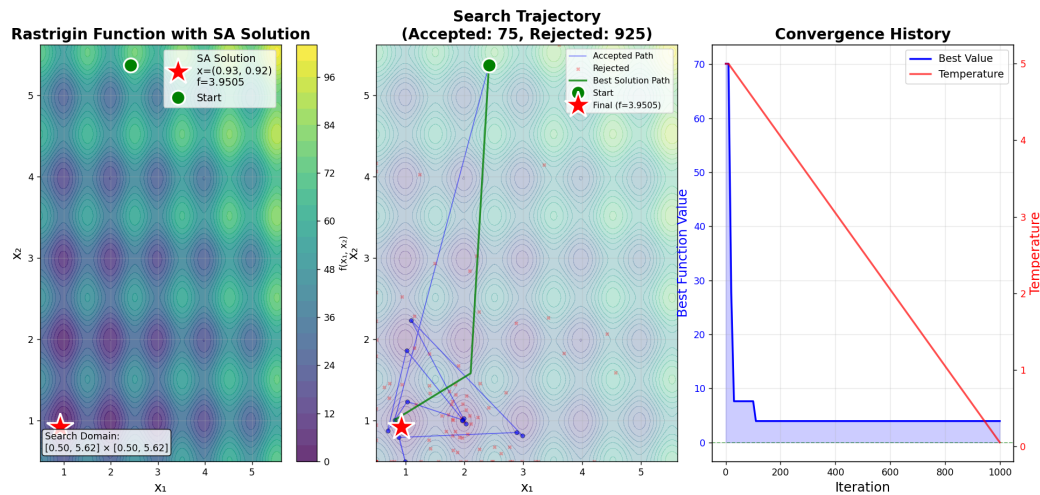
ผลการทดลอง



รูปที่ 1 Rastrigin Function with BnB Solution of bounds $[0.5, 5.62]^2$, $\epsilon = 0.2$

ตารางที่ 1 ผลลัพธ์การหา Global Minimum ด้วย Branch and Bound (BnB)

Domain	ϵ	Solution	$f(x^*)$	Time (s)
2D Offset Bounds $[0.5, 5.62]^2$	0.40	(0.50, 0.50)	2.079	0.003
	0.20	(0.50, 0.50)	2.079	0.005
	0.10	(0.50, 0.50)	2.079	0.019
	0.05	(0.50, 0.50)	2.000	0.049
3D Offset	0.30	$\sim(0.5, 0.5, 0.5)$	3.118	0.053
4D Offset	0.30	$\sim(0.5, 0.5, 0.5)$	4.157	0.695
5D Offset	0.30	$\sim(0.5, 0.5, 0.5)$	5.196	10.332



รูปที่ 2 Rastrigin Function with SA Solution Bounds $[0.5, 5.62]^2$

ตารางที่ 2 ผลลัพธ์การหา Global Minimum ด้วย Simulated Annealing (SA)

Domain	Solution	$f(x^*)$	Time (s)
2D Offset Bounds $[0.5, 5.62]^2$	(0.93, 0.92)	3.951	0.019
	(0.93, 0.92)	3.951	0.022
	(0.93, 0.92)	3.951	0.018
	(0.93, 0.92)	3.951	0.019
3D Offset	$\sim(1.0, 1.0, 1.0)$	3.010	0.028
4D Offset	$\sim(1.0, 1.0, \dots)$	7.029	0.038
5D Offset	$\sim(0.5, 0.5, \dots)$	16.942	0.048

จากผลการทดลองใน Domain $[0.5, 5.62]^d$ พบว่า Branch and Bound (BnB) สามารถหาคำตอบที่ดีที่สุดได้อย่างสม่ำเสมอ โดยมีค่า $f(x^*) = 2.079, 3.118, 4.157$, และ 5.196 สำหรับ 2D, 3D, 4D, และ 5D ตามลำดับ ซึ่งเป็นคำตอบที่ดีที่สุดที่เป็นไปได้ในขอบเขตที่กำหนด ในทางกลับกัน SA ให้คำตอบที่แย่กว่า BnB ในทุกมิติ เช่น ใน 2D มีค่า $f(x^*) = 3.951$ (แย่กว่า BnB 90%) และใน 5D ได้ $f(x^*) = 16.942$ (แย่กว่า BnB 226%) ซึ่งมีสาเหตุหลักมาจาก Search Space ที่ขยายตัวแบบ Exponential แต่จำนวน Iterations คงที่ ทำให้การสำรวจไม่ครบถ้วน โดยเฉพาะบริเวณ Boundary ที่ Optimal Solution มักอยู่

ในด้านเวลาการทำงาน BnB แสดงความเหนือกว่าใน 2D ด้วย $\epsilon = 0.2-0.4$ ที่ใช้เวลาเพียง 0.003-0.019 วินาที ซึ่งเร็วกว่า SA 4-6 เท่า อย่างไรก็ตาม เมื่อมิติเพิ่มขึ้นเป็น 5D BnB ใช้เวลา 10.332 วินาที ซึ่งช้ากว่า SA ถึง 213 เท่า แสดงให้เห็น Exponential Time Complexity $O((L/\epsilon)^d)$ ที่เป็นข้อจำกัดสำคัญ ในทางกลับกัน SA แสดงประสิทธิภาพด้านความเร็วและ Scalability ด้วย Linear Time Complexity $O(d \times k)$ โดยใช้เวลาเพียง 0.019-0.048 วินาทีในทุกมิติ และเวลาเพิ่มขึ้นเพียงประมาณ 0.01 วินาทีต่อมิติ

สรุปผลการทดลอง

การเปรียบเทียบระหว่างอัลกอริทึม Branch and Bound (BnB) และอัลกอริทึม Simulated Annealing (SA) แสดงให้เห็น Trade-off ที่ชัดเจนระหว่างความถูกต้องกับ Scalability ซึ่งในภาพรวม BnB เหมาะสำหรับปัญหาที่มีมิติไม่สูง โดยรับประกันการค้นพบ Global Minimum และให้ผลลัพธ์ที่แม่นยำกว่า SA ในทุกกรณี แต่มีข้อจำกัดด้าน Scalability จาก Exponential Time Complexity และยังต้องการหน่วยความจำที่สูงสำหรับการเก็บ Queue ของ Boxes และการคำนวณ 2^d Corners ต่อ Box ซึ่งทำให้ไม่สามารถใช้กับปัญหาที่มีมิติสูงได้อย่างมีประสิทธิภาพ ในทางกลับกัน SA แสดงจุดเด่นในด้านความเร็วและ Scalability ด้วย Linear Time Complexity ที่คาดการณ์ได้ รวมทั้งยังใช้หน่วยความจำต่ำและ Implement ง่ายด้วย Parameter เพียง 2 ตัว อย่างไรก็ตาม SA มี Accuracy ที่ลดลงอย่างมีนัยสำคัญเมื่อมิติเพิ่มขึ้น และไม่สามารถรับประกันการพบ Global Minimum ได้

จากผลการวิเคราะห์ สามารถสรุปแนวทางการเลือกใช้อัลกอริทึมได้ตามลักษณะปัญหา ดังนี้

1. ปัญหาที่ควรเลือกใช้ Branch and Bound (BnB)
 - ปัญหาที่มีจำนวนมิติไม่เกิน 4 มิติ และยอมรับเวลาในการประมวลผลไม่เกิน 1 วินาที
 - ต้องการความแม่นยำสูงสุดหรือต้องการรับประกันการค้นพบ Global Minimum
 - เป็น Safety-critical Application ที่ไม่สามารถยอมรับคำตอบที่ไม่เหมาะสมได้ เช่น การหลีกเลี่ยง Singularity Configuration ของหุ่นยนต์
 - มี Computational Resource เพียงพอสำหรับการเก็บและประมวลผล Queue
 - ค่า ϵ ที่ยอมรับได้ มีขนาดที่ไม่เล็กเกินไป ($\epsilon \geq 0.2$) เพื่อให้เวลาในการประมวลผลมีความสมเหตุสมผล
 - ฟังก์ชันมี Lower Bound ที่สามารถคำนวณได้อย่างแม่นยำ ทำให้ Pruning มีประสิทธิภาพสูง
2. ปัญหาที่ควรเลือกใช้ Simulated Annealing
 - ปัญหาที่มีจำนวนมิติมากกว่า 4 มิติ ซึ่ง BnB ใช้เวลาในการประมวลผลนานเกินไป (มากกว่า 10 วินาที)
 - ต้องการความเร็วในการตอบสนอง โดยเฉพาะ Real-time Applications ที่ต้องการคำตอบภายใน 0.1 วินาที
 - ยอมรับคำตอบที่มีความเหมาะสมในระดับที่เพียงพอต่อการใช้งาน แทนที่จะเป็นคำตอบที่เหมาะสมที่สุด
 - มีข้อจำกัดด้าน Memory และต้องการอัลกอริทึมที่ใช้ Memory น้อย
 - ต้องการ Implementation ที่ง่ายและรวดเร็ว โดยไม่ต้องออกแบบ Bounding Functions ที่ซับซ้อน
 - สามารถรัน SA หลายครั้ง (Multi-start SA) เพื่อเพิ่มโอกาสในการหาคำตอบที่ดีกว่า

สำหรับการประยุกต์ใช้กับ Motion Planning ของหุ่นยนต์ ผลการศึกษานี้แนะนำให้ใช้ BnB สำหรับหุ่นยนต์ 2-4 DOF ที่ต้องการหลีกเลี่ยง Singularity Configuration อย่างแน่นอน และใช้ SA (พร้อมการปรับปรุง) สำหรับหุ่นยนต์ 6-7 DOF ที่ต้องการความเร็วในการตอบสนองแบบ Real-time โดยอาจต้องใช้ Hybrid Approach ที่รวม SA สำหรับ Global Search กับ Local Optimization สำหรับ Refinement เพื่อสร้างสมดุลระหว่างความเร็วและความแม่นยำ ซึ่งเหมาะสมสำหรับการประยุกต์ใช้ในระบบหุ่นยนต์ที่ต้องการทั้ง 2 คุณสมบัติ